

第七章

语义分析和中间代码产生

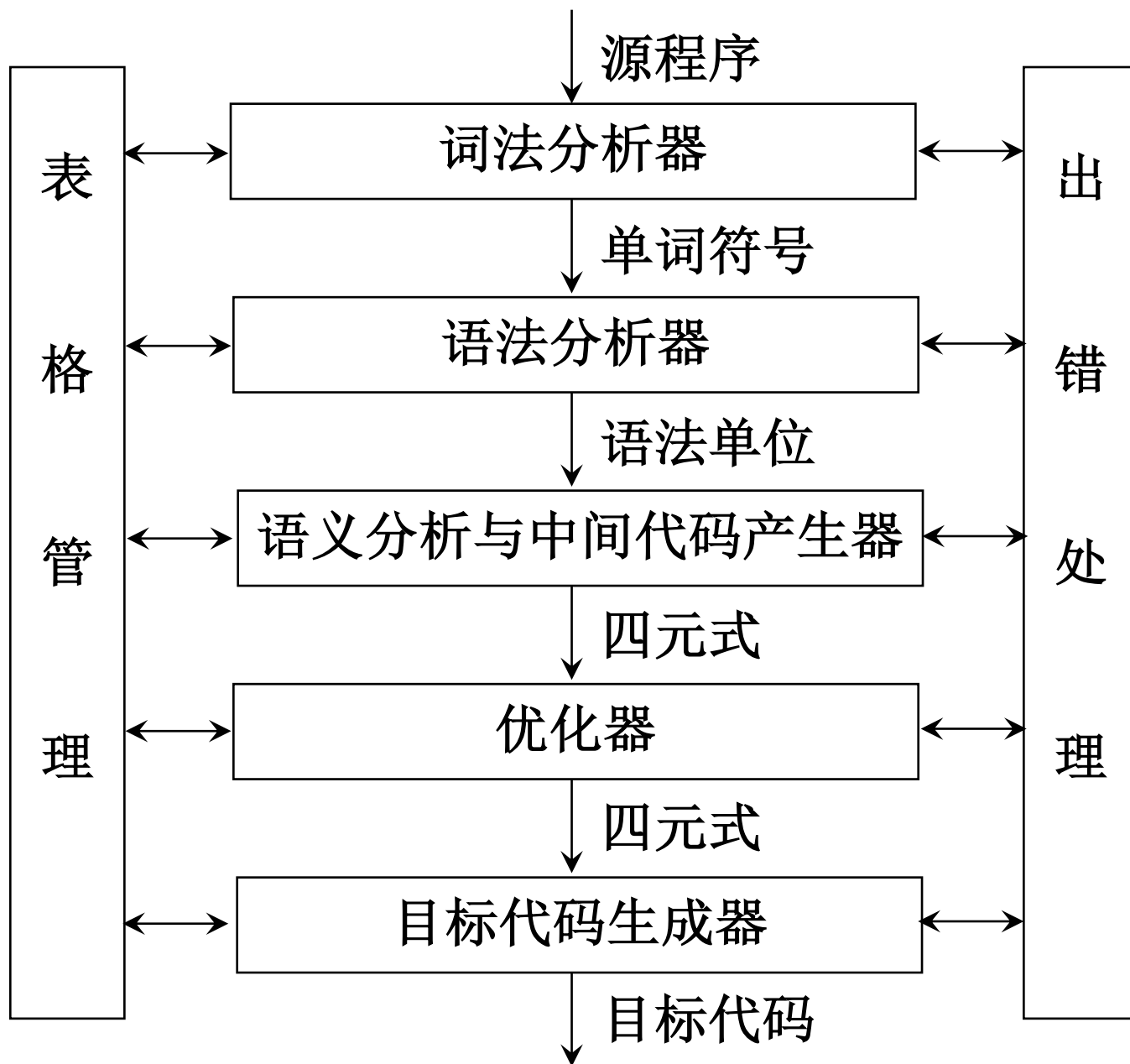


图1.1 编译程序总框

第七章 语义分析和中间代码产生

7.1 中间语言

7.2 说明语句

7.3 赋值语句的翻译

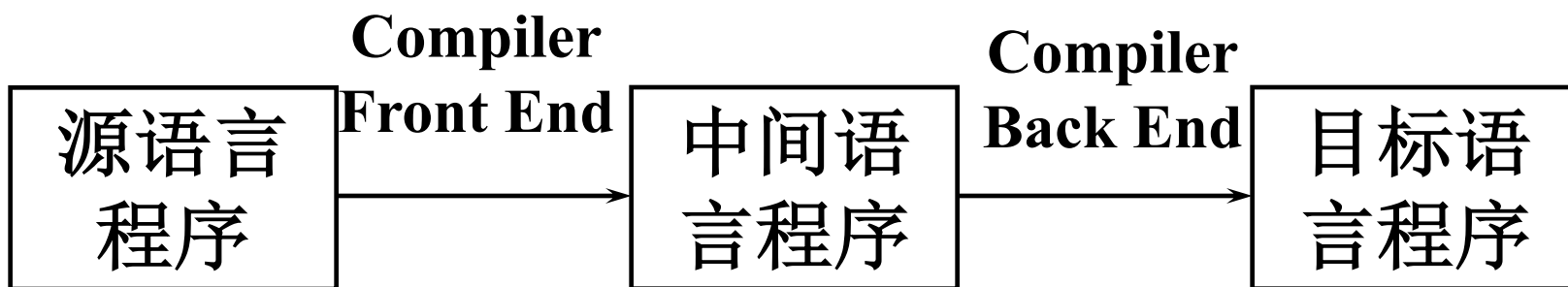
7.4 布尔表达式的翻译

7.5 控制语句的翻译

7.6 过程调用的处理

7.7 类型检查

- 中间语言(复杂性界于源语言和目标语言之间)的好处:
 - 便于进行与机器无关的代码优化工作
 - 易于移植
 - 使编译程序的结构在逻辑上更为简单明确



7.1 中间语言

- 常用的中间语言：
 - 后缀式，逆波兰表示
 - 图表示： DAG、抽象语法树
 - 三地址代码
 - 三元式
 - 四元式
 - 间接三元式

7.1.1 后缀式

- **后缀式**表示法：波兰逻辑学家卢卡西维奇 (Lukasiewicz) 发明的一种表示表达式的方法，又称**逆波兰**表示法。
- 把运算量写在前面，把算符写在后面(后缀)。

如 $a+b$ 写成 $ab+$
 $a*b$ 写成 $ab*$

7.1.1 后缀式

- 一个表达式E的后缀形式E'可以如下定义：
 1. 如果E是一个变量或常量，则E的后缀式是E自身。

E: a E' :a

2. 如果E是 $E_1 \text{ op } E_2$ 形式的表达式，其中op是任何二元操作符，则E的后缀式为 $E_1' E_2' \text{ op}$ ，其中 E_1' 和 E_2' 分别为 E_1 和 E_2 的后缀式。

E: a+b E' :ab+

3. 如果E是 (E_1) 形式的表达式，则E的后缀式是 E_1 的后缀式。

E: (a) E' :a

7.1.1 后缀式

- 逆波兰表示法不用括号。

如: $(a+b)*c$

表示为: $(a+b)c^* \Rightarrow ab+bc^*$

只要知道每个算符的目数，对于后缀式，不论从哪一端进行扫描，都能对它进行唯一分解。

如： 后綴式 所代表的表达式

$$ab+c^* \quad (a+b)c^* \quad \Rightarrow \quad (a+b)^*c$$
$$abc+^* \quad a(b+c)^* \quad \Rightarrow \quad a^*(b+c)$$
$$\begin{aligned} ab+cd+^* & \quad (a+b)cd+^* \Rightarrow (a+b)(c+d)^* \\ & \Rightarrow (a+b)^*(c+d) \end{aligned}$$

7.1.1 后缀式

表7.1 把表达式翻译成后缀式的语义规则描述

产生式	语义规则
$E \rightarrow E_1 \text{ op } E_2$	$E.\text{code} := E_1.\text{code} E_2.\text{code} \text{op}$
$E \rightarrow (E_1)$	$E.\text{code} := E_1.\text{code}$
$E \rightarrow \text{id}$	$E.\text{code} := \text{id}$

$E.\text{code}$ 表示E后缀形式

op表示任意二元操作符

“||”表示后缀形式的连接

7.1.2 图表示法

- 图表示法
 - DAG
 - 抽象语法树

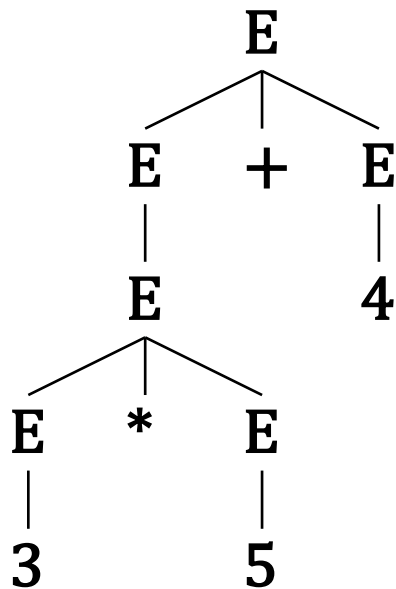
6.2.4 抽象语法树(p144)

- 在语法树中去掉那些对翻译不必要的信息，从而获得更有效的源程序中间表示。这种经变换后的语法树称之为**抽象语法树**(Abstract Syntax Tree)
- 在抽象语法树中，**操作符和关键字都不作为叶结点出现，而是把它们作为内部结点，即叶结点的父结点。**

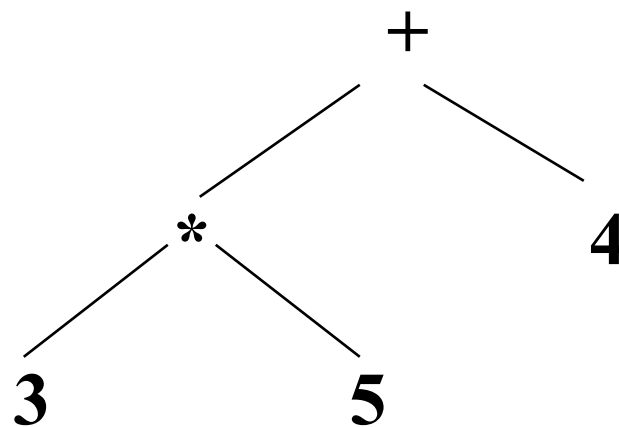
6.2.4 抽象语法树

□ $3*5+4$

$E \rightarrow E + E \mid E * E \mid \text{digit}$



语法树

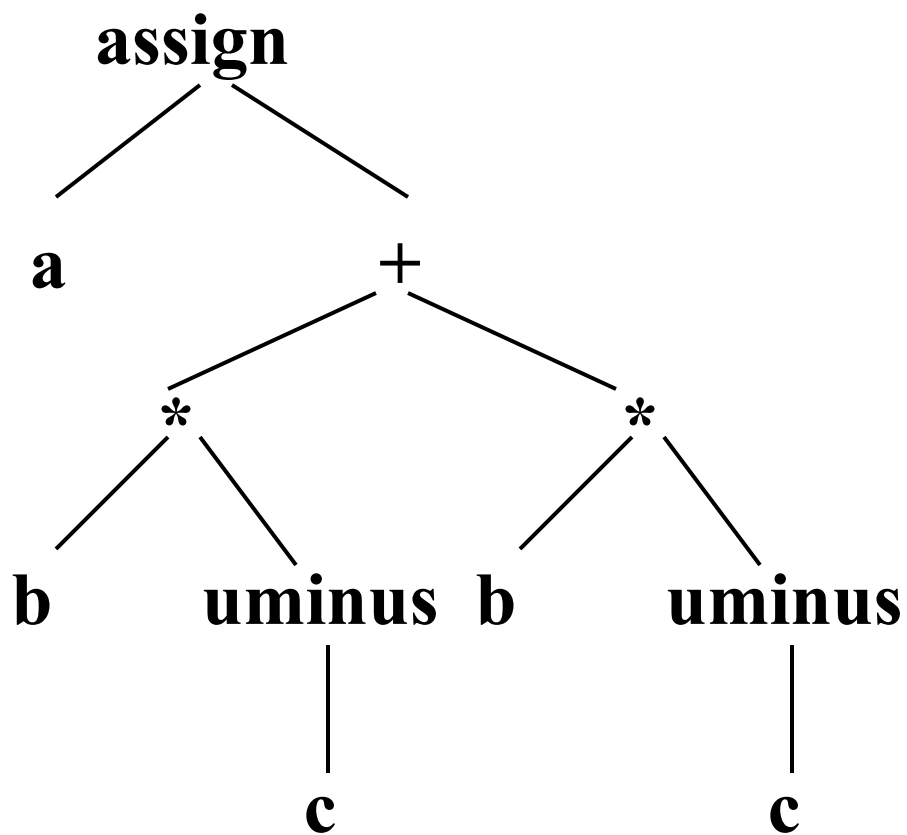


抽象语法树

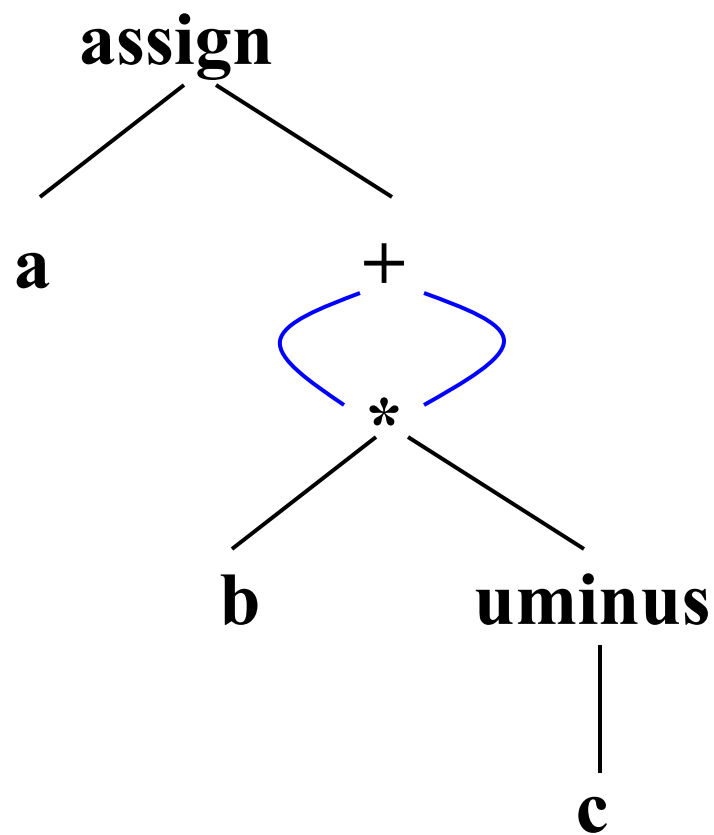
7.1.2 图表示法

- 无循环有向图(Directed Acyclic Graph, 简称DAG)
 - 对表达式中的每个子表达式, DAG中都有一个结点
 - 一个内部结点代表一个操作符, 它的孩子代表操作数
 - 在DAG中代表公共子表达式的结点具有多个父结点, 在抽象语法树中公共子表达式被表示为重复的子树。

$a := b * -c + b * -c$ 的图表示法



抽象语法树



DAG

产生赋值语句抽象语法树的属性文法

产生式

语义规则

$S \rightarrow id := E$	$S.nptr := mknode('assign',$ $mkleaf(id, id.place), E.nptr)$
$E \rightarrow E_1 + E_2$	$E.nptr := mknode('+', E_1.nptr, E_2.nptr)$
$E \rightarrow E_1 * E_2$	$E.nptr := mknode('*', E_1.nptr, E_2.nptr)$
$E \rightarrow -E_1$	$E.nptr := mknode('uminus', E_1.nptr)$
$E \rightarrow (E_1)$	$E.nptr := E_1.nptr$
$E \rightarrow id$	$E.nptr := mkleaf(id, id.place)$

$a := b * -c + b * -c$

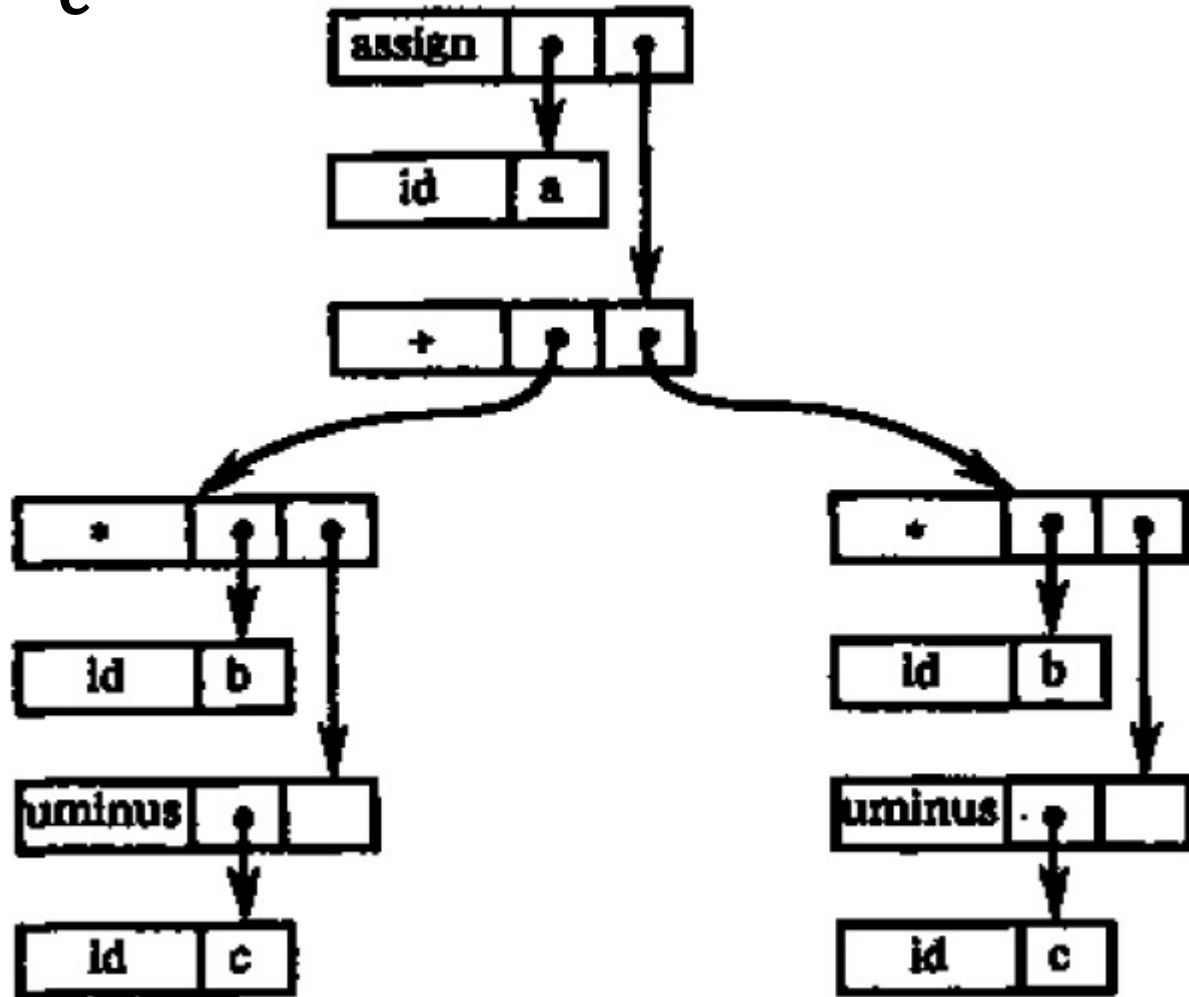


图7.4 抽象语法树的表示方法

7.1.3 三地址代码

- 三地址代码

$x := y \text{ op } z$

其中,

x 、 y 、 z : 名字、常数或编译时产生的临时变量;

op : 运算符号。

每个语句的右边只能有一个运算符。

如: 源语言表达式 $x+y*z$

被翻译为: $t_1 := y*z$

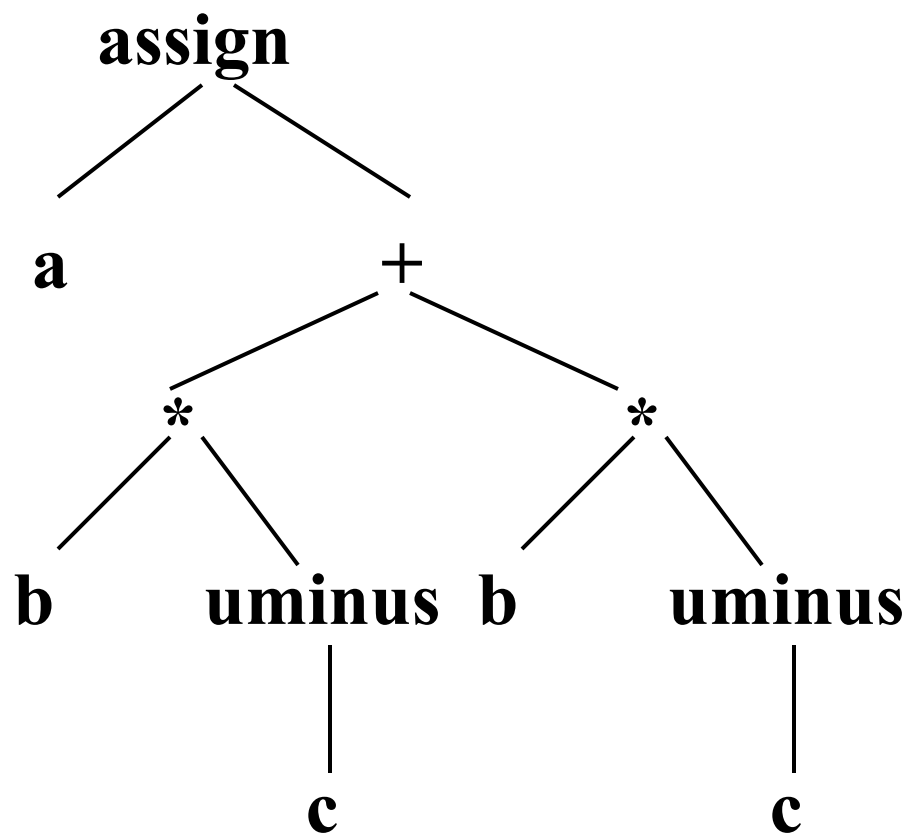
$t_2 := x+t_1$

其中, t_1 、 t_2 为编译时产生的临时变量。

7.1.3 三地址代码

- 三地址代码可以看成是抽象语法树或DAG的一种线性表示
- 生成三地址代码时，临时变量的名字对应抽象语法树的内部结点

a:=b*-c+b*-c



抽象语法树

抽象语法树对应的代码

T₁ := -c

T₂ := b * T₁

T₃ := -c

T₄ := b * T₃

T₅ := T₂ + T₄

a := T₅

7.1.3 三地址代码

- 每条语句通常包含三个地址，两个用来表示操作数，一个用来存放结果。
- 用户定义的名字在实际实现时将由指向符号表中的相应名字入口的指针所代替。
- 三地址语句类似于汇编语言代码。语句可以带有符号标号，而且存在各种控制流语句。符号标号代表存放中间代码的数组中三地址代码语句的下标。

三地址语句的种类

- $x := y \text{ op } z$
- $x := \text{op } y$
- $x := y$
- `goto L`
- `if x relop y goto L`或`if a goto L`
- `param x`和`call p,n`, 以及返回语句`return y`
- $x := y[i]$ 及 $x[i] := y$ 的索引赋值
- $x := \&y$, $x := *y$ 和 $*x := y$ 的地址和指针赋值

为赋值语句生成三地址代码的S-属性文法定义

$S \rightarrow id := E$

$E \rightarrow E_1 + E_2$

$E \rightarrow E_1 * E_2$

$E \rightarrow -E_1$

$E \rightarrow (E_1)$

$E \rightarrow id$

- 非终结符号S有综合属性**S.code**，它代表赋值语句S的三地址代码。
- 非终结符号E有如下两个综合属性：
 - **E.place**表示存放E值的名字。
 - **E.code**表示对E求值的三地址语句序列。
 - 函数**newtemp()**的功能是，每次调用它时，将返回一个不同临时变量名字,如T₁,T₂,...。

表7.3 对赋值语句产生三地址代码的S-属性文法

$S \rightarrow id := E$	$S.code := E.code \parallel \text{gen}(id.place \text{ ':=' } E.place)$
$E \rightarrow E_1 + E_2$	$E.place := \text{newtemp}();$ $E.code := E_1.code \parallel E_2.code \parallel$ $\text{gen}(E.place \text{ ':=' } E_1.place \text{ '+' } E_2.place)$
$E \rightarrow E_1 * E_2$	$E.place := \text{newtemp}();$ $E.code := E_1.code \parallel E_2.code \parallel$ $\text{gen}(E.place \text{ ':=' } E_1.place \text{ '*' } E_2.place)$
$E \rightarrow -E_1$	$E.place := \text{newtemp}();$ $E.code := E_1.code \parallel$ $\text{gen}(E.place \text{ ':=' 'uminus' } E_1.place)$
$E \rightarrow (E_1)$	$E.place := E_1.place;$ $E.code := E_1.code$
$E \rightarrow id$	$E.place := id.place;$ $E.code := ' '$

7.1.3 三地址代码

- 三地址代码语句的实现可以用记录表示，记录中包含表示运算符和操作数的域。
- 三种表示方法：
 - 四元式
 - 三元式
 - 间接三元式

7.1.3 三地址代码

- 四元式
 - 一个带有四个域的记录结构，这四个域分别称为op, arg1, arg2及result

三地址语句

四元式

	<u>op</u>	<u>arg1</u>	<u>arg2</u>	<u>result</u>
x := y op z	op	y	z	x
x := -y	uminus	y		x
x := y	assign	y		x

三地址语句

$T_1 := -c$

$T_2 := b * T_1$

$T_3 := -c$

$T_4 := b * T_3$

$T_5 := T_2 + T_4$

$a := T_5$

	op	arg1	arg2	result
(0)	uminus	c		T_1
(1)	*	b	T_1	T_2
(2)	uminus	c		T_3
(3)	*	b	T_3	T_4
(4)	+	T_2	T_4	T_5
(5)	assign	T_5		a

四元式中的arg1, arg2和result的内容都是一个指针，此指针指向有关名字的符号表入口。
临时变量名也要填入符号表。

7.1.3 三地址代码

- 三元式
 - 避免把临时变量填入到符号表，通过计算这个临时变量值的语句的**位置**来引用这个临时变量。
 - 三个域： op, arg1, arg2。
 - 运算符op的两个操作数域arg1和arg2，或者是指向符号表的指针（名字或常量），或者是指向三元式表的指针（临时变量）。

三地址语句

$T_1 := -c$
 $T_2 := b * T_1$
 $T_3 := -c$
 $T_4 := b * T_3$
 $T_5 := T_2 + T_4$
 $a := T_5$

	op	arg1	arg2
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	assign	a	(4)

三地址语句

- 间接三元式
 - 为了便于优化，用 三元式表+间接码表 表示中间代码
 - 间接码表:一张指示器表，按运算的先后次序列出有关三元式在三元式表中的位置。
 - 优点: 方便优化，节省空间

- 例如，语句

$$X := (A + B) * C;$$

$$Y := D \uparrow (A + B)$$

的间接三元式表示如下表所示。

间接代码

三元式表

		OP	ARG1	ARG2
(1)	(1)	+	A	B
(2)	(2)	*	(1)	C
(3)	(3)	:=	X	(2)
(4)	(4)	↑	D	(1)
(5)	(5)	:=	Y	(4)

7.2 说明语句

- 说明语句的翻译：对每个局部名字，在符号表中建立相应的表项，并填入有关的信息如类型、在存储器中的相对地址等。

7.2.1 过程中的说明语句

在C、Pascal及FORTRAN等语言的语法中，允许在一个过程中的所有说明语句作为一个组来处理，把它们安排在一所数据区中。

需要一个[全程变量](#)如offset来跟踪下一个可用的相对地址的位置。

类型说明的文法和翻译模式

$P \rightarrow D$

$D \rightarrow D; D$

$D \rightarrow \text{id} : T$

$T \rightarrow \text{integer}$

$T \rightarrow \text{real}$

$T \rightarrow \text{array}[\text{num}] \text{ of } T_1$

$T \rightarrow \uparrow T_1$

`x:integer;y:real`

类型说明的文法和翻译模式

$P \rightarrow \{ \text{offset} := 0 \} D$

$D \rightarrow D; D$

$D \rightarrow \text{id} : T$ $\{ \text{enter}(\text{id.name}, T.\text{type}, \text{offset});$
 $\text{offset} := \text{offset} + T.\text{width} \}$

$T \rightarrow \text{integer}$ $\{ T.\text{type} := \text{integer};$
 $T.\text{width} := 4 \}$

$T \rightarrow \text{real}$ $\{ T.\text{type} := \text{real};$
 $T.\text{width} := 8 \}$

$T \rightarrow \text{array}[\text{num}] \text{ of } T_1$ $\{ T.\text{type} := \text{array}(\text{num.val}, T_1.\text{type});$
 $T.\text{width} := \text{num.val} * T_1.\text{width} \}$

$T \rightarrow \uparrow T_1$ $\{ T.\text{type} := \text{pointer}(T_1.\text{type});$
 $T.\text{width} := 4 \}$

x:integer;y:real

类型说明的文法和翻译模式

$P \rightarrow MD$

$M \rightarrow \epsilon$ { offset := 0 }

$D \rightarrow D;D$

$D \rightarrow id:T$ { enter(id.name, T.type, offset);
 offset := offset + T.width }

$T \rightarrow integer$ { T.type := integer;
 T.width := 4 }

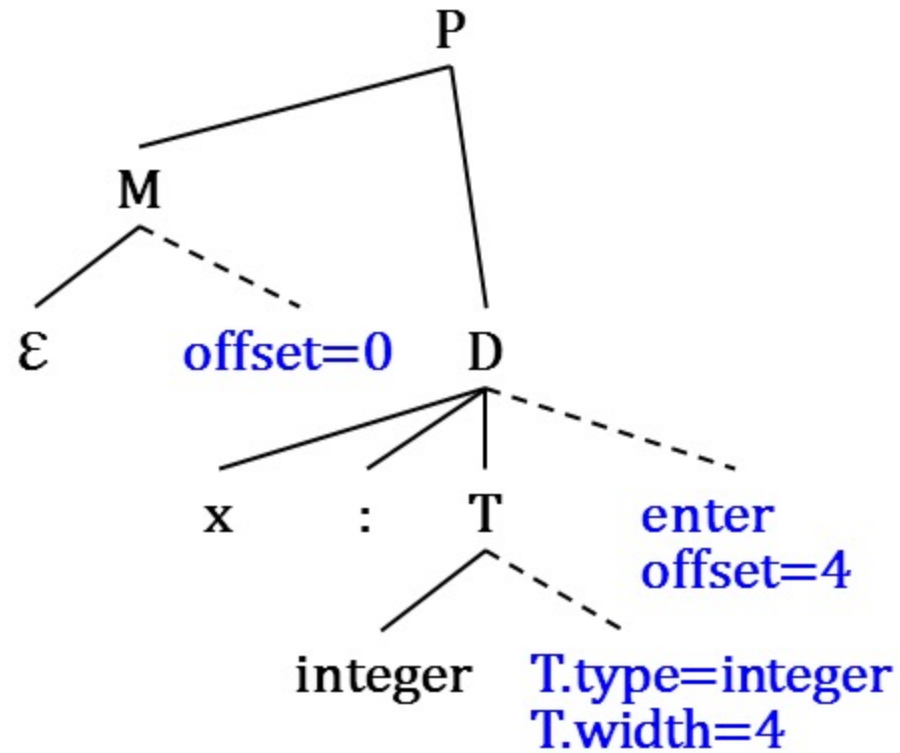
$T \rightarrow real$ { T.type := real;
 T.width := 8 }

$T \rightarrow array[num] \text{ of } T_1$ { T.type := array(num.val, T₁.type);
 T.width := num.val * T₁.width }

$T \rightarrow \uparrow T_1$ { T.type := pointer(T₁.type);
 T.width := 4 }

x:integer;y:real

x:integer



7.3 赋值语句的翻译

- 7.3.1 简单算术表达式及赋值语句
- 7.3.2 数组元素的引用

7.3.1 简单算术表达式及赋值语句

- 属性和过程
 - **id.name**: id所代表的名字本身;
 - **lookup(id.name)**: 检查是否在符号表中存在相应此名字的入口,
如果有, 则返回一个指向该表项的指针;
否则, 返回nil表示没有找到。
 - **emit()**: 将生成的三地址语句发送到输入文件中;
 - **E.place**: 符号表项的指针;
 - **newtemp()**: 返回一个不同临时变量名字, 如 $T_1, T_2, \dots$, 把该名字也存入符号表, 并返回符号表项的指针。

图7.10 产生赋值语句三地址代码的翻译模式

$S \rightarrow id := E$	{ p := lookup(id.name); if p ≠ nil then emit(p := E.place) else error }
$E \rightarrow E_1 + E_2$	{ E.place := newtemp(); emit(E.place := E ₁ .place + E ₂ .place) }
$E \rightarrow E_1 * E_2$	{ E.place := newtemp(); emit(E.place := E ₁ .place * E ₂ .place) }
$E \rightarrow -E_1$	{ E.place := newtemp(); emit(E.place := 'uminus' E ₁ .place) }
$E \rightarrow (E_1)$	{ E.place := E ₁ .place }
$E \rightarrow id$	{ p := lookup(id.name); if p ≠ nil then E.place := p else error }

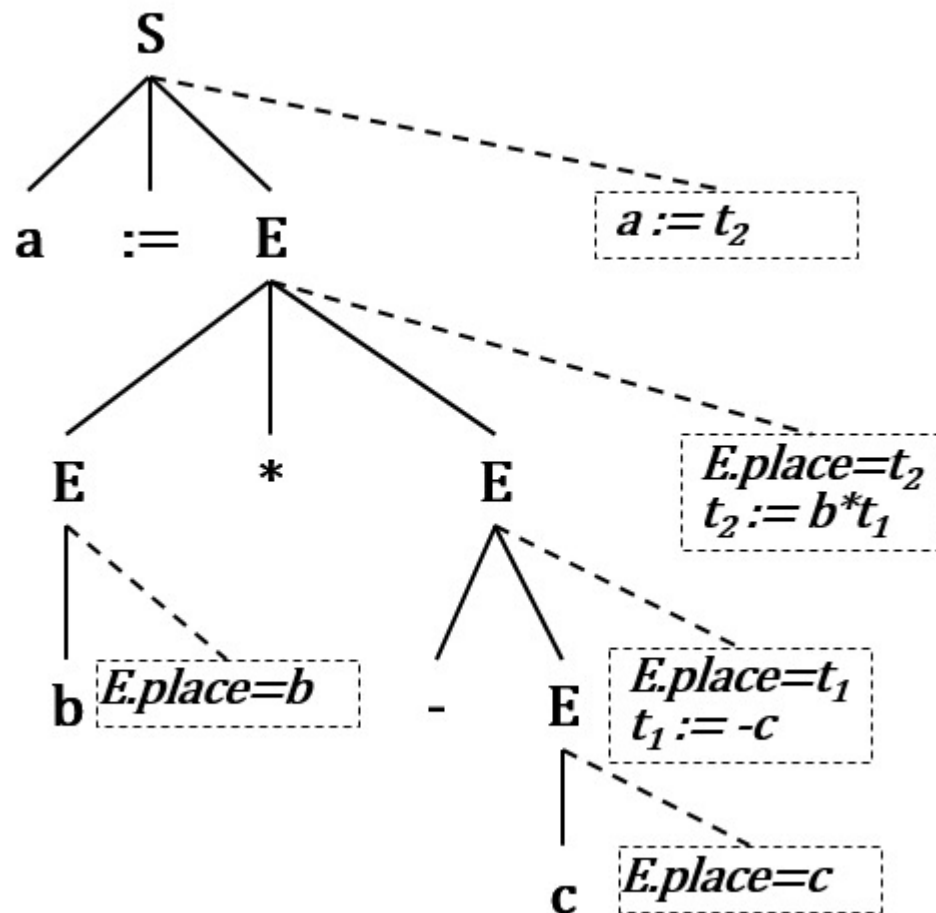
输入: $a := b * -c$

三地址代码:

$t_1 := -c$

$t_2 := b + t_1$

$a := t_2$



7.3.2 数组元素的引用

- 数组元素地址的计算

- 一维数组

设数组A每个元素宽度为w，

A[i]的相对地址：

$$\text{base} + (i - \text{low}) * w \quad (7.4)$$

其中：

low: 数组下标的下界；

base: A的第一个元素A[low]的相对地址。

$$\Rightarrow i * w + (\text{base} - \text{low} * w)$$

$$\Rightarrow i * w + (\text{base} - C)$$

其中：

$C = \text{low} * w$ ，编译时计算并存放在符号表中数组A的对应项中。

7.3.2 数组元素的引用

- 数组元素地址的计算

- 二维数组-按行存放

$A[i_1, i_2]$ 的相对地址:

$$\text{base} + ((i_1 - \text{low}_1) * n_2 + i_2 - \text{low}_2) * w$$

其中:

low_1 、 low_2 : 分别为 i_1 、 i_2 的下界;

n_2 : i_2 可取值的个数。

$$\Rightarrow (i_1 * n_2 + i_2) * w + (\text{base} - (\text{low}_1 * n_2 + \text{low}_2) * w) \quad (7.5)$$

$$\Rightarrow (i_1 * n_2 + i_2) * w + (\text{base} - C)$$

其中:

$C = (\text{low}_1 * n_2 + \text{low}_2) * w$, 编译时计算并存放在符号表中数组A 的对应项中。

A[1,1]
A[1,2]
A[1,3]
A[2,1]
A[2,2]
A[2,3]

7.3.2 数组元素的引用

- 数组元素地址的计算
 - 多维数组-按行存放

$A[i_1, i_2, \dots, i_k]$ 的相对地址:

$$((\dots((i_1 n_2 + i_2) n_3 + i_3) \dots) n_k + i_k) \times w +$$

$$\text{base} - ((\dots((\text{low}_1 n_2 + \text{low}_2) n_3 + \text{low}_3) \dots) n_k + \text{low}_k) \times w$$

$$\Rightarrow ((\dots i_1 n_2 + i_2) n_3 + i_3) \dots n_k + i_k) \times w + (\text{base} - C)$$

其中:

$C = ((\dots((\text{low}_1 n_2 + \text{low}_2) n_3 + \text{low}_3) \dots) n_k + \text{low}_k) \times w$, 编译时计算并存放在符号表中数组A 的对应项中。

7.3.2 数组元素的引用

- 数组元素地址的计算与数组引用的文法联系起来
- `id`出现的地方也允许下面产生式中的`L`出现

$$L \rightarrow \text{Elist }] \mid \text{id}$$
$$\text{Elist} \rightarrow \text{Elist}, E \mid \text{id } [E$$

在整个下标表达式串`Elist`的翻译过程中都能知道符号表中相应于数组名字`id`的全部信息。

- 地址计算变量部分-式(7.6)的第一个子项元素 $A[i_1, i_2, i_3, \dots, i_k]$ 的相对地址:

$$((\dots((i_1 n_2 + i_2) n_3 + i_3) \dots) n_k + i_k) \times w +$$

$$\text{base} - ((\dots((\text{low}_1 n_2 + \text{low}_2) n_3 + \text{low}_3) \dots) n_k + \text{low}_k) \times w$$

递归计算:

$$e_1 = i_1,$$

$$e_2 = e_1 * n_2 + i_2,$$

$$e_3 = e_2 * n_3 + i_3,$$

.....

$$e_k = e_{k-1} * n_k + i_k$$

递归计算:

$$e_k = e_{k-1} * n_k + i_k$$

$L \rightarrow \text{Elist }] \mid \text{id}$

$\text{Elist} \rightarrow \text{Elist}, E \mid \text{id} [E$

- 语义变量或语义过程:
 - Elist.array: 符号表中相应数组名字表项的指针;
 - Elist.ndim: 下标表达式的个数, 即维数;
 - limit(array,j): 返回 n_j , 即array的第j维的长度;
 - Elist.place: 存放由Elist中的下标表达式计算出来的值。

$L \rightarrow E \text{ list }] \mid \text{id}$

$E \text{ list} \rightarrow E \text{ list}, E \mid \text{id} [E$

- L的左值

- L.place:

- 若L为简单名字，L.place为符号表中相应此名字表项的指针；
 - 若L为数组引用，数组计算中的常量部分。

- L.offset :

- 若L为简单名字，L.place为null；
 - 若L为数组引用，数组计算中的变量部分。

- (1) $S \rightarrow L := E$
- (2) $E \rightarrow E + E$
- (3) $E \rightarrow (E)$
- (4) $E \rightarrow L$
- (5) $L \rightarrow Elist \]$
- (6) $L \rightarrow id$
- (7) $Elist \rightarrow Elist, E$
- (8) $Elist \rightarrow id \ [\ E$

- (1) $S \rightarrow L := E$
- (2) $E \rightarrow E + E$
- (3) $E \rightarrow (E)$
- (4) $E \rightarrow L$
- (5) $L \rightarrow Elist \]$
- (6) $L \rightarrow id$
- (7) $Elist \rightarrow Elist, E$
- (8) $Elist \rightarrow id \ [\ E$

(8) $Elist \rightarrow id[E$
 { $Elist.place := E.place;$
 $Elist.ndim := 1;$
 $Elist.array := id.place \ }$

- (1) $S \rightarrow L := E$
- (2) $E \rightarrow E + E$
- (3) $E \rightarrow (E)$
- (4) $E \rightarrow L$
- (5) $L \rightarrow \text{Elist }]$
- (6) $L \rightarrow \text{id}$
- (7) $\text{Elist} \rightarrow \text{Elist}, E$
- (8) $\text{Elist} \rightarrow \text{id } [E$

(7) $\text{Elist} \rightarrow \text{Elist}_1, E$

```
{  t:=newtemp( );
    m:=Elist1.ndim+1;
    emit(t ':=' Elist1.place '*' limit(Elist1.array,m) );
    emit(t ':=' t '+' E.place);
    Elist.array:= Elist1.array;
    Elist.place:=t;
    Elist.ndim:=m
}
```

元素 $A[i_1, i_2, i_3, \dots, i_k]$ 的相对地址:

$((\dots((i_1 n_2 + i_2) n_3 + i_3) \dots) n_k + i_k) \times w +$

$\text{base} - ((\dots((\text{low}_1 n_2 + \text{low}_2) n_3 + \text{low}_3) \dots) n_k + \text{low}_k) \times w$

(5) $L \rightarrow \text{Elist}$]

```
{ L.place := newtemp();  
  emit(L.place ':= ' Elist.array '—' C);  
  L.offset := newtemp();  
  emit(L.offset ':= ' w '*' Elist.place) }
```

(6) $L \rightarrow \text{id}$ { L.place := id.place; L.offset := null }

- (1) $S \rightarrow L := E$
- (2) $E \rightarrow E + E$
- (3) $E \rightarrow (E)$
- (4) $E \rightarrow L$
- (5) $L \rightarrow \text{Elist}$]
- (6) $L \rightarrow \text{id}$
- (7) $\text{Elist} \rightarrow \text{Elist}, E$
- (8) $\text{Elist} \rightarrow \text{id}$ [E

- (1) $S \rightarrow L := E$
- (2) $E \rightarrow E + E$
- (3) $E \rightarrow (E)$
- (4) $E \rightarrow L$
- (5) $L \rightarrow Elist$
- (6) $L \rightarrow id$
- (7) $Elist \rightarrow Elist, E$
- (8) $Elist \rightarrow id [E$

(1) $S \rightarrow L := E$

{ if $L.offset = null$ then /*L是简单变量*/

emit($L.place := E.place$)

else emit($L.place [L.offset] := E.place$)}

(4) $E \rightarrow L$

{ if L.offset=null then

$E.place := L.place$

else begin

$E.place := \text{newtemp}()$;

 emit($E.place :=$ **$L.place$** **$[$** **$L.offset$** **$]$**)

end

}

(1) $S \rightarrow L := E$

(2) $E \rightarrow E + E$

(3) $E \rightarrow (E)$

(4) $E \rightarrow L$

(5) $L \rightarrow \text{Elist}$]

(6) $L \rightarrow \text{id}$

(7) $\text{Elist} \rightarrow \text{Elist}, E$

(8) $\text{Elist} \rightarrow \text{id}$ [E

- (1) $S \rightarrow L := E$
- (2) $E \rightarrow E + E$
- (3) $E \rightarrow (E)$
- (4) $E \rightarrow L$
- (5) $L \rightarrow \text{Elist }]$
- (6) $L \rightarrow \text{id}$
- (7) $\text{Elist} \rightarrow \text{Elist}, E$
- (8) $\text{Elist} \rightarrow \text{id } [E$

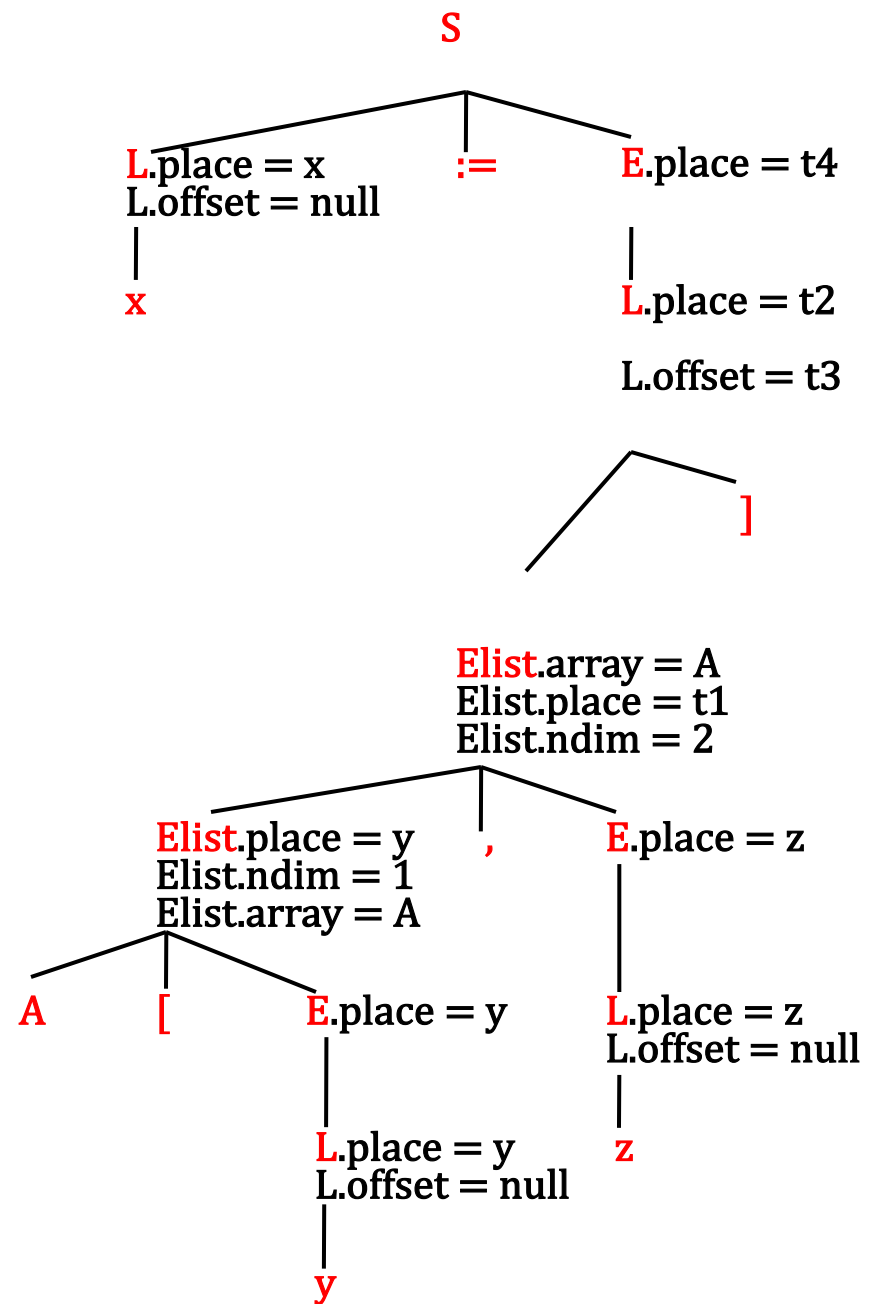
(2) $E \rightarrow E_1 + E_2$
 { $E.\text{place} := \text{newtemp}()$;
 $\text{emit}(E.\text{place} := E_1.\text{place} + E_2.\text{place})$ }

(3) $E \rightarrow (E_1) \{ E.\text{place} := E_1.\text{place} \}$

设A: n1=10, n2=20, w=4,
low1=low2=1
赋值语句: $x := A[y, z]$

三地址代码:

$t1 = y * 20$
 $t1 = t1 + z$
 $t2 = A - 84$
 $t3 = 4 * t1$
 $t4 = t2[t3]$
 $x = t4$



设A: n1=10, n2=20, w=4,
low1=low2=1
赋值语句: $x := A[y, z]$

三地址代码:

t1 = y*20
t1 = t1+z
t2=A-84
t3=4*t1
t4=t2[t3]
x=t4

变址取数

四元式:

100 (*, y, 20, t1)
101 (+, t1, z, t1)
102 (-, A, 84, t2)
103 (*, 4, t1, t3)
104 ([], t2[t3], -, t4)
105 (assign, t4, -, x)

变址存数:

t2[t3]=t4
([]=, t4, -, t2[t3])

类型转换

- 混合运算：一个表达式中可能出现各种不同类型的变量或常数
- **E.type**: 表示非终结符E的类型属性

$$\begin{aligned} E \rightarrow E_1 \text{ op } E_2 \\ \quad \{ \text{if } E_1.\text{type} = \text{integer and } E_2.\text{type} = \text{integer} \\ \quad \quad E.\text{type} := \text{integer} \\ \quad \text{else } E.\text{type} := \text{real} \} \end{aligned}$$

例:

$y + i * j$

其中y为实型，i、j为整型

三地址代码:

$T_1 := i \text{ int} * j$

$T_3 := \text{inttoreal } T_1$

$T_2 := y \text{ real} + T_3$

$E \rightarrow E_1 + E_2$

{ E.place:=newtemp();

if $E_1.type = \text{integer}$ and $E_2.type = \text{integer}$ then begin

emit(E.place ':=' $E_1.place$ 'int+' $E_2.place$);

E.type:=integer

end

else if $E_1.type = \text{real}$ and $E_2.type = \text{real}$ then begin

emit(E.place ':=' $E_1.place$ 'real+' $E_2.place$);

E.type:=real

end

```

else if E1.type=integer and E2.type=real then begin
    u:=newtemp( );
    emit (u ':=' 'inttoreal' E1.place);
    emit (E.place ':=' u 'real+' E2.palce);
    E.type:=real
end
else if E1.type=real and E1.type=integer then begin
    u:=newtemp( );
    emit (u ':=' 'inttoreal' E2.place);
    emit (E.place ':=' E1.place 'real+' u);
    E.type:=real
end
else E.type:=type_error}

```

7.4 布尔表达式的翻译

- 7.4.1 数值表示法
- 7.4.2 作为条件控制的布尔式翻译

7.4 布尔表达式的翻译

- 布尔表达式的两个基本作用：
 - 计算逻辑值
 - 控制流语句中的条件表达式
- 产生布尔表达式的文法：

$E \rightarrow E \text{ or } E \mid E \text{ and } E \mid \text{not } E \mid (E) \mid \text{id relop id} \mid \text{id}$

relop.op: 确定relop指的是6个关系运算符中的哪一个: $<, \leq, ==, !=, >, \geq$

结合性: or和and左结合

优先级: $\text{or} < \text{and} < \text{not}$

- 计算布尔表达式通常采用两种方法:

1. 如同计算算术表达式一样,一步步算

$$\begin{aligned} & 1 \text{ or } (\text{not } 0 \text{ and } 0) \text{ or } 0 \\ &= 1 \text{ or } (1 \text{ and } 0) \text{ or } 0 \\ &= 1 \text{ or } 0 \text{ or } 0 \\ &= 1 \text{ or } 0 \\ &= 1 \end{aligned}$$

2. 采用某种优化措施

A or B:	if A then true else B
A and B:	if A then B else false
not A:	if A then false else true

7.4.1 数值表示法

- 布尔表达式:
 a or b and not c
- 三地址语句序列:
 T1:=not c
 T2:=b and T1
 T3:=a or T2

7.4.1 数值表示法

- 关系表达式:

$a < b$

可等价地写成

if $a < b$ then 1 else 0

三地址语句序列:

100: if $a < b$ goto 103

101: T:=0

102: goto 104

103: T:=1

104:

- 布尔表达式的数值表示法的翻译模式
 - 过程emit(): 将三地址代码送到输出文件中
 - 变量nextstat: 给出输出序列中下一条三地址语句的地址索引
 - 每产生一条三地址语句后, 过程emit()便把nextstat加1

关于布尔表达式的数值表示法的翻译模式

$E \rightarrow E_1 \text{ or } E_2$ $\{E.place := newtemp();$
 $emit(E.place := E_1.place \text{ 'or' } E_2.place)\}$

$E \rightarrow E_1 \text{ and } E_2$ $\{E.place := newtemp();$
 $emit(E.place := E_1.place \text{ 'and' } E_2.place)\}$

$E \rightarrow \text{not } E_1$ $\{E.place := newtemp();$
 $emit(E.place := \text{'not' } E_1.place)\}$

$E \rightarrow (E_1)$ $\{E.place := E_1.place\}$

$E \rightarrow id_1 \text{ relop } id_2$
 $\{E.place := newtemp();$
 $emit(\text{'if' } id_1.place \text{ relop. op } id_2.place$
 $\text{'goto' nextstat+3});$
 $emit(E.place := \text{'0'});$
 $emit(\text{'goto' nextstat+2});$
 $emit(E.place := \text{'1'}) \}$

$E \rightarrow id$ $\{E.place := id.place\}$

$a < b$

100: if $a < b$ goto 103
101: $T_1 := 0$
102: goto 104
103: $T_1 := 1$

$a < b$ or $c < d$ and $e < f$

$E \rightarrow E_1$ or E_2 $\{E.place := newtemp();$
 $emit(E.place := E_1.place \text{ 'or' } E_2.place)\}$

$E \rightarrow E_1$ and E_2 $\{E.place := newtemp();$
 $emit(E.place := E_1.place \text{ 'and' } E_2.place)\}$

$E \rightarrow \text{not } E_1$ $\{E.place := newtemp();$
 $emit(E.place := \text{ 'not' } E_1.place)\}$

$E \rightarrow (E_1)$ $\{E.place := E_1.place\}$

$E \rightarrow id_1$ relop id_2
 $\{E.place := newtemp();$
 $emit(\text{ 'if' } id_1.place \text{ relop. op } id_2.place$
 $\text{ 'goto' nextstat+3});$
 $emit(E.place := \text{ '0'});$
 $emit(\text{ 'goto' nextstat+2});$
 $emit(E.place := \text{ '1'}) \}$

$E \rightarrow id$ $\{E.place := id.place\}$

100: if $a < b$ goto 103
101: $T_1 := 0$
102: goto 104
103: $T_1 := 1$
104: if $c < d$ goto 107
105: $T_2 := 0$
106: goto 108
107: $T_2 := 1$
108: if $e < f$ goto 111
109: $T_3 := 0$
110: goto 112
111: $T_3 := 1$
112: $T_4 := T_2$ and T_3
113: $T_5 := T_1$ or T_4

7.4.2 作为条件控制的布尔式翻译

条件语句

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

布尔表达式 E 控制对 S_1 和 S_2 的选择。

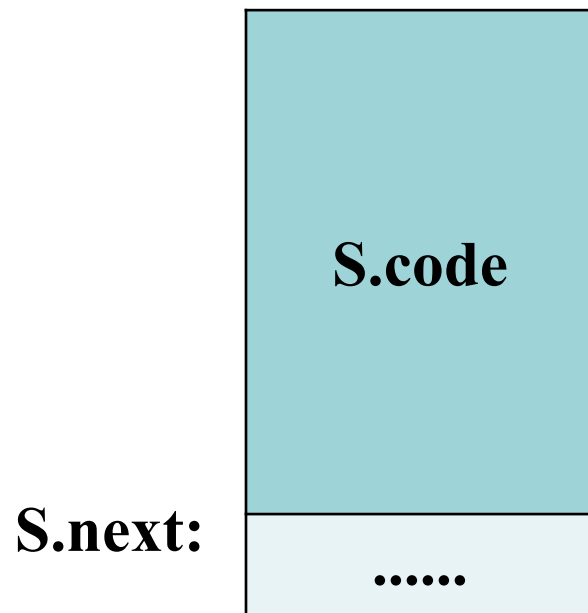


图7.15 if-then-else语句的代码结构

7.4.2 作为条件控制的布尔式翻译

条件语句

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

布尔表达式 E 控制对 S_1 和 S_2 的选择。

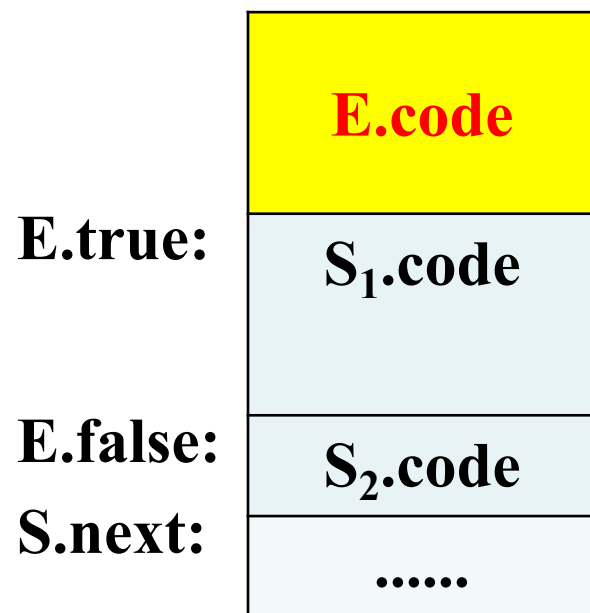


图7.15 if-then-else语句的代码结构

7.4.2 作为条件控制的布尔式翻译

条件语句

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

布尔表达式 E 控制对 S_1 和 S_2 的选择

- 赋予它两种“出口”：
 - “真”出口：出向 S_1
 - “假”出口：出向 S_2
- 翻译为一串跳转指令

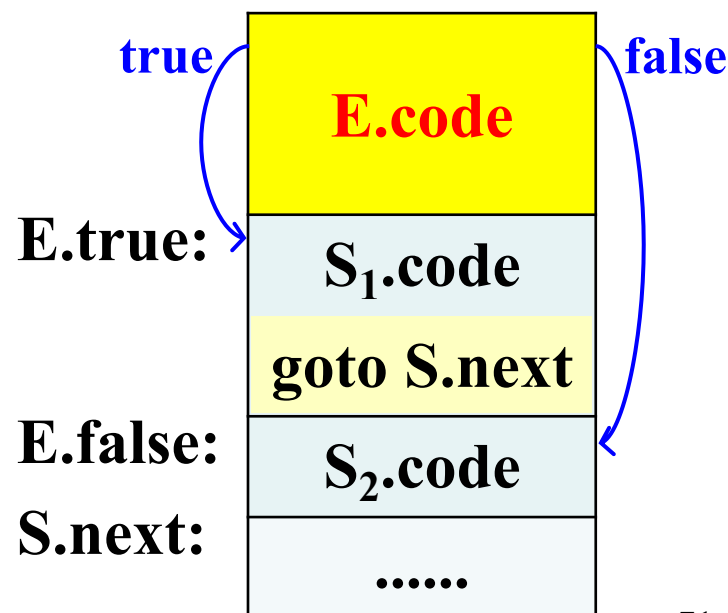


图7.15 if-then-else语句的代码结构

7.4.2 作为条件控制的布尔式翻译

条件语句

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

布尔表达式 E 控制对 S_1 和 S_2 的选择

```
if E goto E.true
goto E.false
E.true: S1
      goto S.next
E.false:      S2
S.next:
```

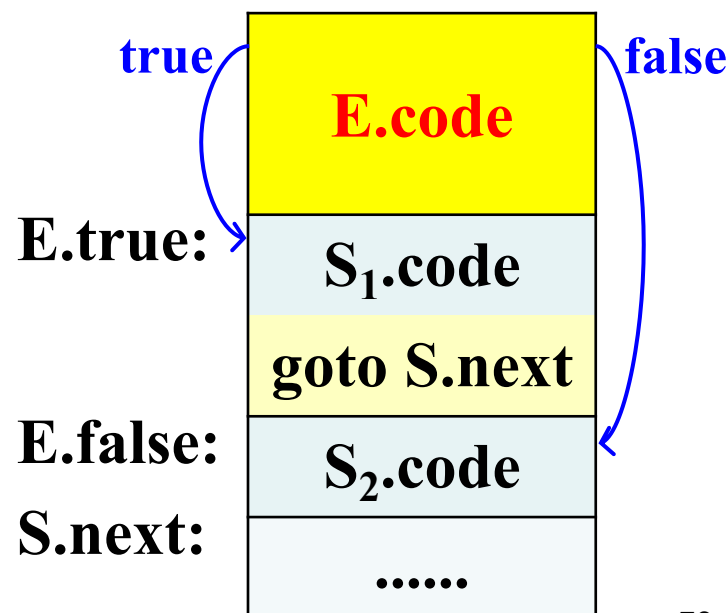


图7.15 if-then-else语句的代码结构

7.4.2 作为条件控制的布尔式翻译

- 作为转移条件的布尔式，把它翻译为一串**跳转指令**。
- 继承属性
 - E.true: E为真时控制流转向的标号
 - E.false: E为假时控制流转向的标号
- 函数
 - newlabel(): 返回一个新的符号标号

产生布尔表达式三地址代码的语义规则

产生式

$E \rightarrow E_1 \text{ or } E_2$

$E \rightarrow E_1 \text{ and } E_2$

$E \rightarrow \text{not } E_1$

$E \rightarrow (E_1)$

$E \rightarrow \text{id}_1 \text{ relop id}_2$

$E \rightarrow \text{true}$

$E \rightarrow \text{false}$

产生式 语义规则

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$E \rightarrow id_1 \text{ relop } id_2$

$E.code :=$

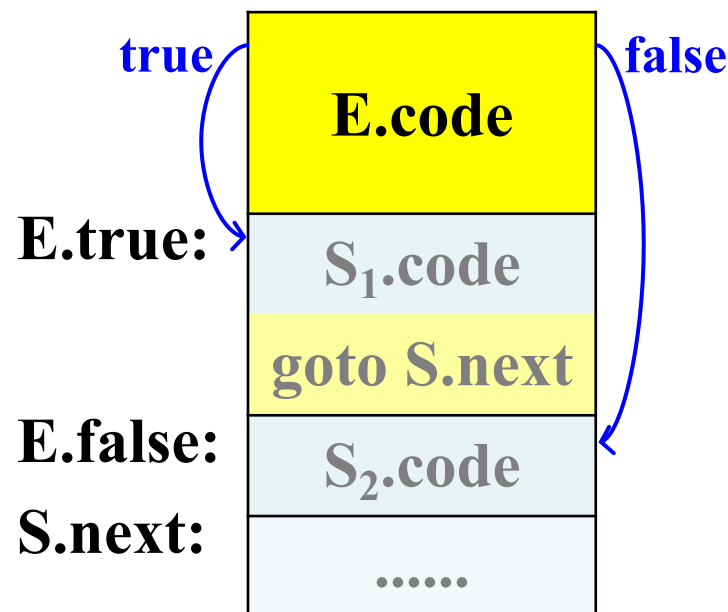
gen('if' $id_1.place$ relop.op $id_2.place$

'goto' E.true) ||

gen('goto' E.false)

$a > c$

if $a > c$ goto Etrue
goto Efalse



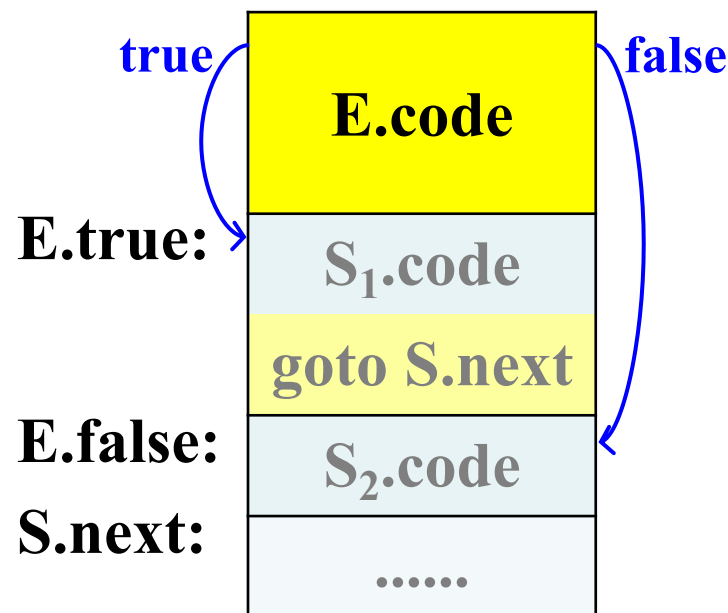
三地址代码

图7.15 if-then-else语句的代码结构

产生式 语义规则

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$E \rightarrow E_1 \text{ or } E_2$



三地址代码

图7.15 if-then-else语句的代码结构

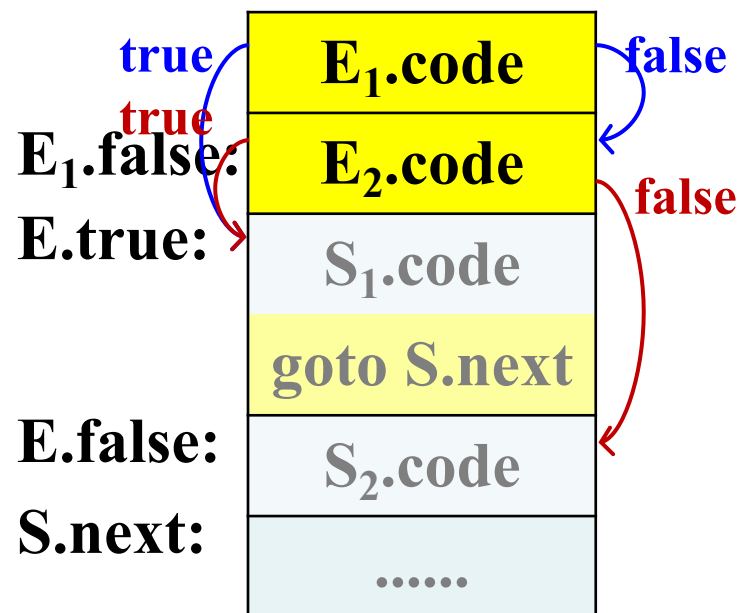
产生式 语义规则

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$E \rightarrow E_1 \text{ or } E_2$	$E_1.\text{true} := E.\text{true};$ $E_1.\text{false} := \text{newlabel}();$ $E_2.\text{true} := E.\text{true};$ $E_2.\text{false} := E.\text{false};$ $E.\text{code} := E_1.\text{code} \parallel \text{gen}(E_1.\text{false} ':') \parallel E_2.\text{code}$
-------------------------------------	--

$a > c \text{ or } b < d$

	if $a > c$ goto Etrue
	goto L1
L1:	if $b < d$ goto Etrue
	goto Efalse



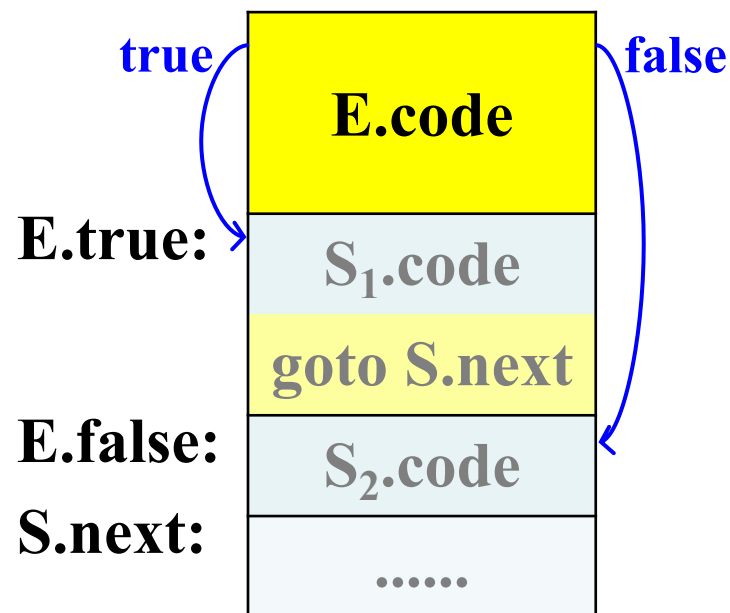
三地址代码

图 $E \rightarrow E_1 \text{ or } E_2$ 的代码结构

产生式 语义规则

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$E \rightarrow E_1 \text{ and } E_2$



三地址代码

图7.15 if-then-else语句的代码结构

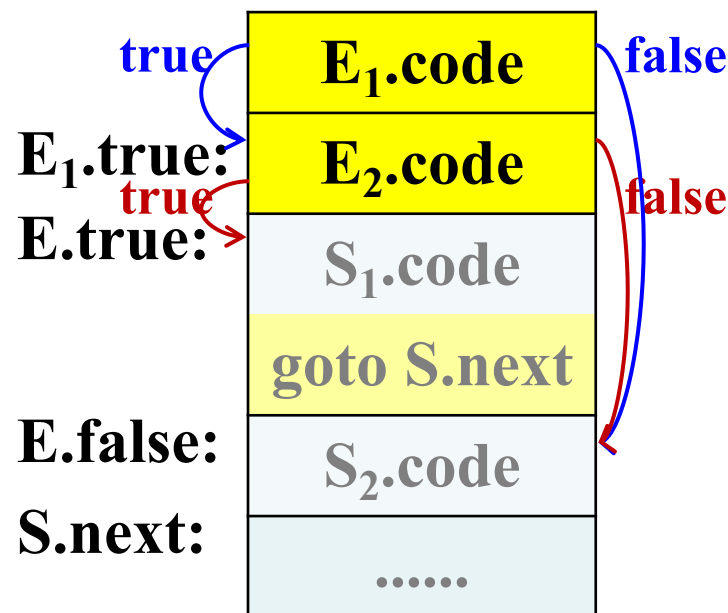
产生式 语义规则

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$E \rightarrow E_1 \text{ and } E_2$	$E_1.\text{true} := \text{newlabel}();$ $E_1.\text{false} := E.\text{false};$ $E_2.\text{true} := E.\text{true};$ $E_2.\text{false} := E.\text{false};$ $E.\text{code} := E_1.\text{code} \parallel \text{gen}(E_1.\text{true} ':') \parallel E_2.\text{code}$
--------------------------------------	--

$a > c \text{ and } b < d$

	if $a > c$ goto L1
	goto Efalse
L1:	if $b < d$ goto Etrue
	goto Efalse



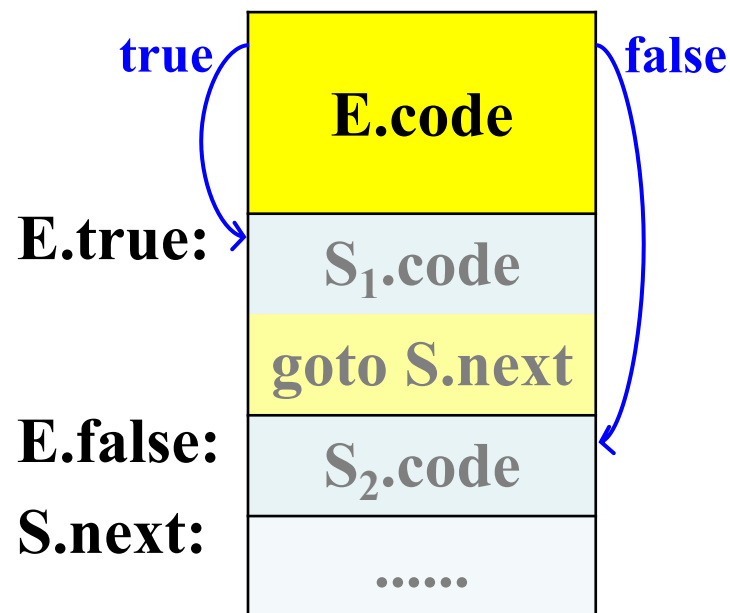
三地址代码

图 $E \rightarrow E_1 \text{ and } E_2$ 的代码结构

产生式 语义规则

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$E \rightarrow \text{not } E_1$



三地址代码

图7.15 if-then-else语句的代码结构

产生式 语义规则

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$E \rightarrow \text{not } E_1$	$E_1.\text{true} := E.\text{false};$ $E_1.\text{false} := E.\text{true};$ $E.\text{code} := E_1.\text{code}$
---------------------------------	--

not a>c

if a>c goto Efalse goto Etrue

三地址代码

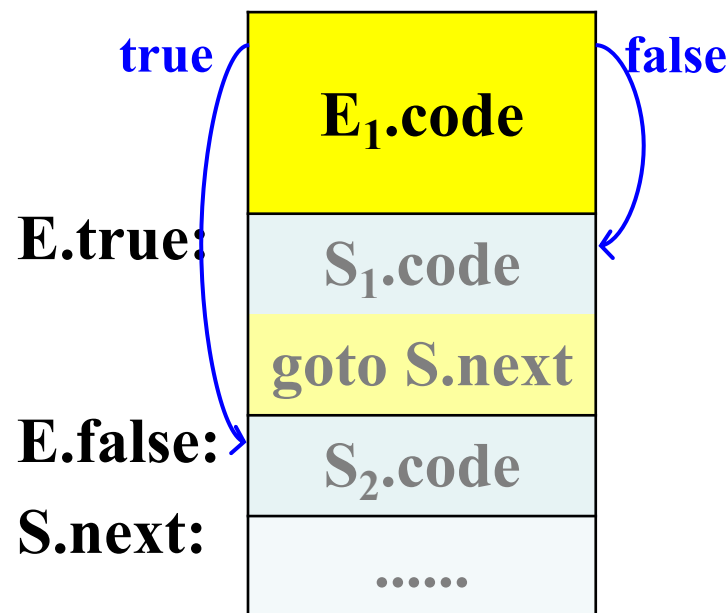
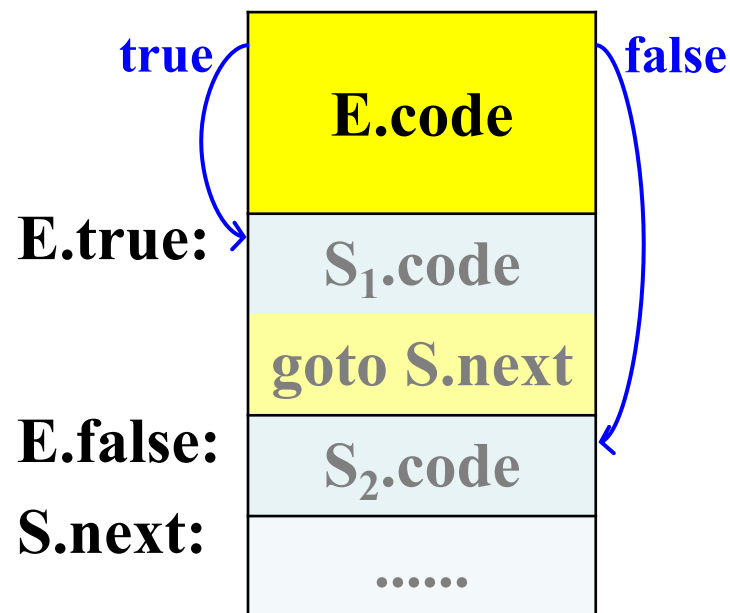


图 $E \rightarrow \text{not } E_1$ 的代码结构

产生式 语义规则

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$E \rightarrow (E_1)$



三地址代码

图7.15 if-then-else语句的代码结构

产生式 语义规则

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$E \rightarrow (E_1)$	$E_1.\text{true} := E.\text{true};$ $E_1.\text{false} := E.\text{false};$ $E.\text{code} := E_1.\text{code}$
-----------------------	--

(a>c)

if a>c goto Etrue goto Efalse

三地址代码

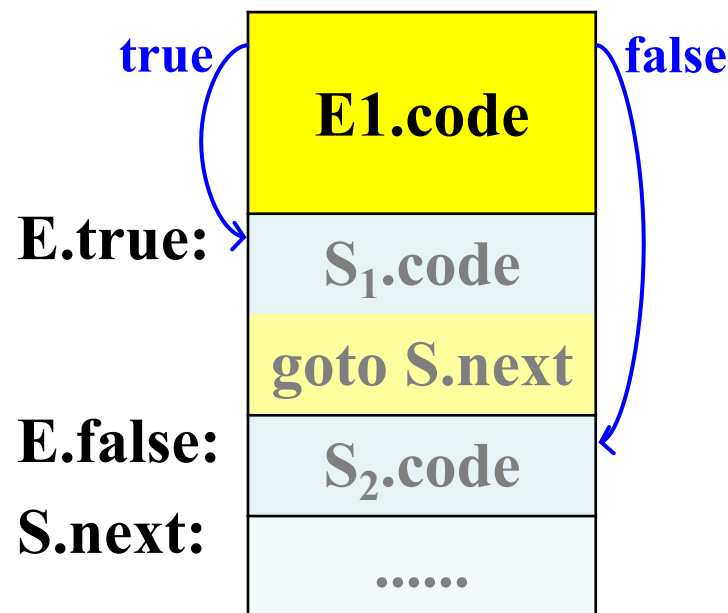


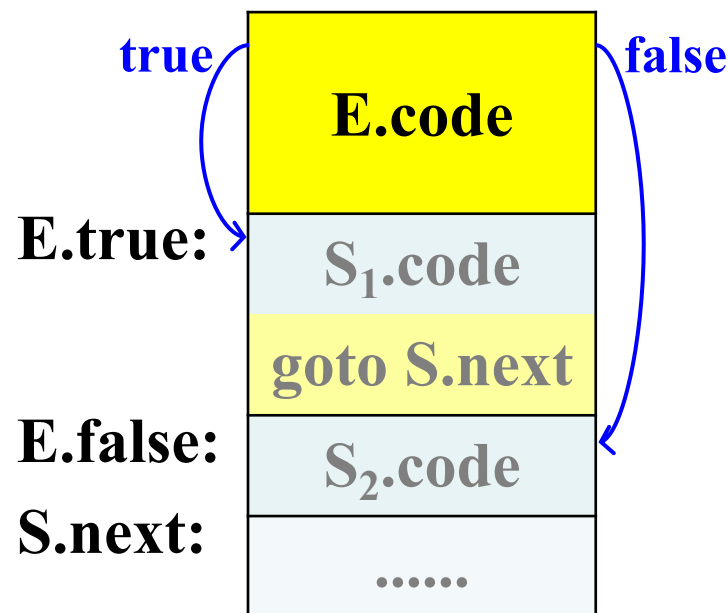
图 $E \rightarrow (E_1)$ 的代码结构

产生式 语义规则

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$E \rightarrow \text{true}$ $E.\text{code} := \text{gen}(\text{'goto' } E.\text{true})$

$E \rightarrow \text{false}$ $E.\text{code} := \text{gen}(\text{'goto' } E.\text{false})$



三地址代码

图7.15 if-then-else语句的代码结构

例7.3:

$a < b$ or $\triangle c < d$ and $\triangle e < f$

假定整个表达式的真假出口已分别置为Ltrue和Lfalse

```
                if a < b goto Ltrue
                goto L1
L1:             if c < d goto L2
                goto Lfalse
L2:             if e < f goto Ltrue
                goto Lfalse
```

例7.3:

$a < b$ or $\triangle c < d$ and $\triangle e < f$

假定整个表达式的真假出口已分别置为Ltrue和Lfalse

```
100:      if a<b goto Ltrue
101:      goto L1 102
102: L1:   if c<d goto L2 104
103:      goto Lfalse
104: L2:   if e<f goto Ltrue
105:      goto Lfalse
```

7.4.2 作为条件控制的布尔式翻译

- 四元式
 - 把四元式存入一个数组中，数组下标就代表四元式的标号。

三地址代码

if a goto p

if x rop y goto p

goto p

四元式

(jnz, a, -, p)

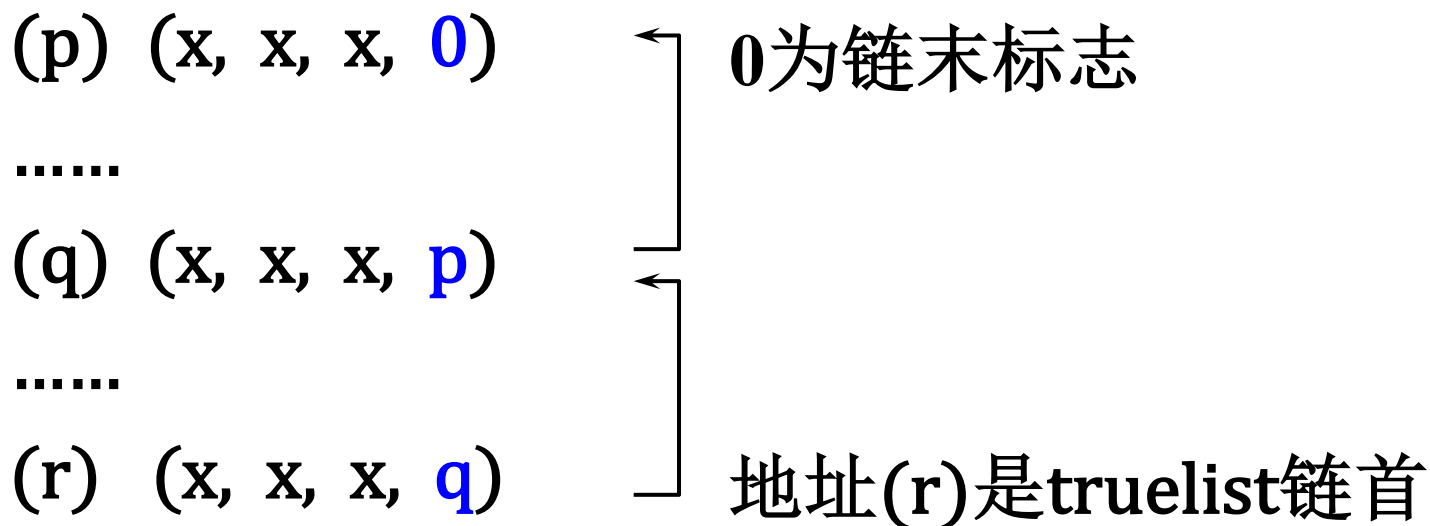
(jrop, x, y, p)

(j, -, -, p)

7.4.2 作为条件控制的布尔式翻译

- 一遍扫描
 - 问题：当生成某些转移语句时，可能还不知道该语句将要转移到的标号是什么。
 - 回填技术
 - 生成形式分支的跳转指令时暂时不确定跳转目标；
 - 建立一个链表，把转向这个目标的跳转指令的标号键入这个链表；
 - 目标确定之后再把它填入有关的跳转指令中。

- 为非终结符E赋予两个综合属性**E.truelist**和**E.falselist**。它们分别记录布尔表达式E所对应的四元式中需回填“真”、“假”出口的四元式的标号所构成的链表。
- 借助需要回填的跳转四元式的第四区段构造链表。
- 例如:假定E的四元式中需回填“真”出口的有p、q和r三个四元式，则“真”链E.truelist为:



- 变量或函数（过程）：

- 变量 **nextquad**，指向下一条将要产生但尚未形成的四元式的地址(标号)。nextquad的初值为1，每当执行一次emit之后，nextquad将增1。
- 函数 **makelist(i)**，将创建一个仅含i的新链表，其中i是四元式数组的一个下标(标号)；函数返回指向这个链的指针。
- 函数 **merge(p1,p2)**，把以p1和p2为链首的两条链合并为一，作为函数值，回送合并后的链首。
- 过程 **backpatch(p, t)**，其功能是完成“回填”，把p所链接的每个四元式的第四区段都填为t。

7.4.2 作为条件控制的布尔式翻译

- (1) $E \rightarrow E_1 \text{ or } M E_2$
- (2) | $E_1 \text{ and } M E_2$
- (3) | $\text{not } E_1$
- (4) | (E_1)
- (5) | $\text{id}_1 \text{ relop id}_2$
- (6) | id
- (7) $M \rightarrow \varepsilon$

(5) $E \rightarrow id_1 \text{ relop } id_2$
 { $E.truelist := makelist(nextquad);$
 $E.falselist := makelist(nextquad+1);$
 $emit('j \text{ relop.op } ', id_1.place, ' ', id_2.place, ' ', '0');$
 $emit('j, -, -, 0')$ }

(1) $E \rightarrow E_1 \text{ or } M E_2$
 { $backpatch(E_1.falselist, M.quad);$
 $E.truelist := merge(E_1.truelist, E_2.truelist);$
 $E.falselist := E_2.falselist$ }

(2) $E \rightarrow E_1 \text{ and } M E_2$
 { $backpatch(E_1.truelist, M.quad);$
 $E.truelist := E_2.truelist;$
 $E.falselist := merge(E_1.falselist, E_2.falselist)$ }

(7) $M \rightarrow \varepsilon$ { $M.quad := nextquad$ }

(3) $E \rightarrow \text{not } E_1$

```
{ E.truelist:=E1.falselist;  
  E.falselist:=E1.truelist}
```

(4) $E \rightarrow (E_1)$

```
{ E.truelist:=E1.truelist;  
  E.falselist:=E1. falselist}
```

(6) $E \rightarrow \text{id}$

```
{ E.truelist:=makelist(nextquad);  
  E.falselist:=makelist(nextquad+1);  
  emit('jnz' , id.place , '-' , '0');  
  emit(' j, -, -, 0') }
```

$a < b$ or $c < d$ and $e < f$

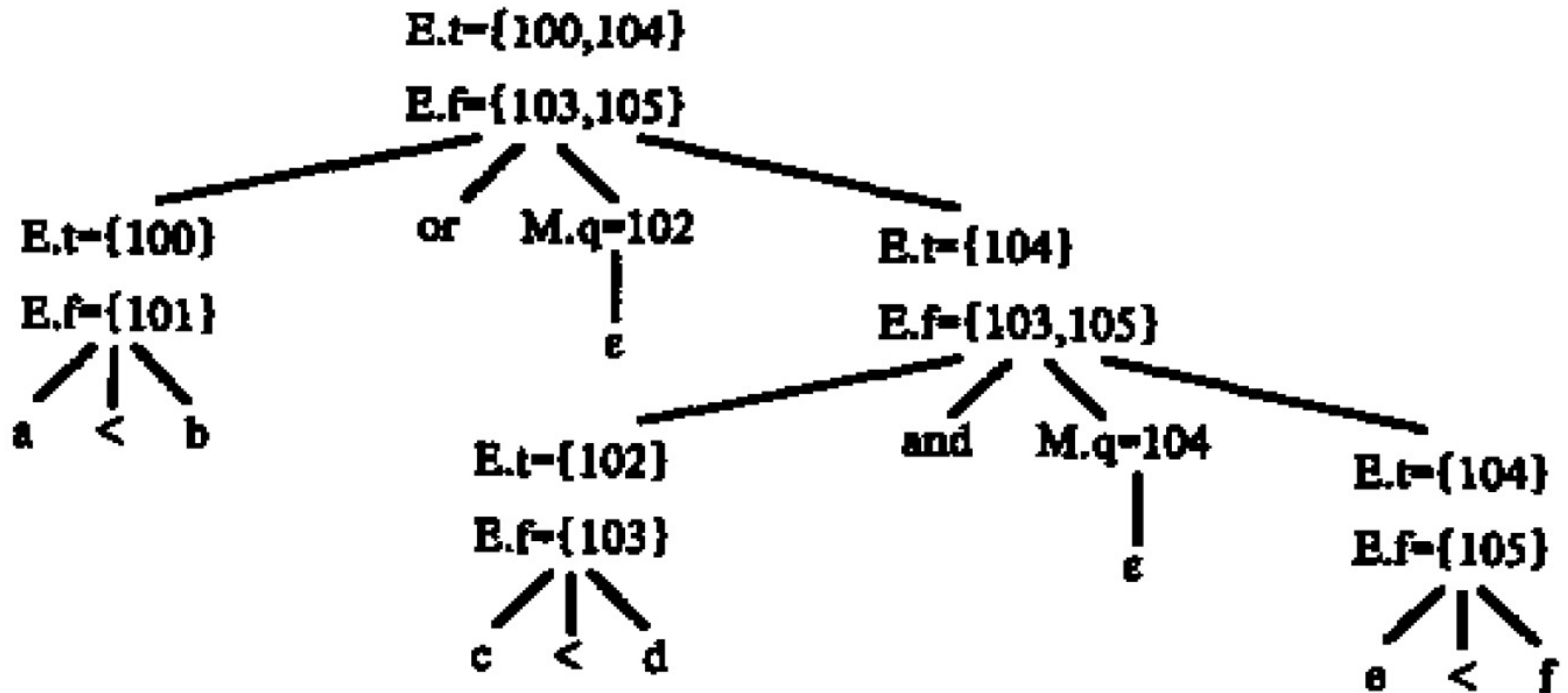


图 7.16 关于 $a < b$ or $c < d$ and $e < f$ 的加了注释的分析树

7.4.2 作为条件控制的布尔式翻译

- 四元式

$a < b$ or $c < d$ and $e < f$

四元式:

100 (j<, a, b, 0)

101 (j, -, -, 102)

102 (j<, c, d, 104)

103 (j, -, -, 0)

104 (j<, e, f, 100)

105 (j, -, -, 103)

7.5 控制语句的翻译

控制流语句

$$\begin{aligned} S \rightarrow & \text{ if } E \text{ then } S_1 \\ & | \text{ if } E \text{ then } S_1 \text{ else } S_2 \\ & | \text{ while } E \text{ do } S_1 \end{aligned}$$

其中，E为布尔表达式。

继承属性S.next: 继S的代码之后将被执行的第一条三地址指令。

7.5.1 控制流语句

产生式

语义规则

$S \rightarrow \text{if } E \text{ then } S_1$

$E.\text{true} := \text{newlabel}();$

$E.\text{false} := S.\text{next};$

$S_1.\text{next} := S.\text{next};$

$S.\text{code} := E.\text{code} \parallel$

$\text{gen}(E.\text{true} ':')$

$S_1.\text{code}$

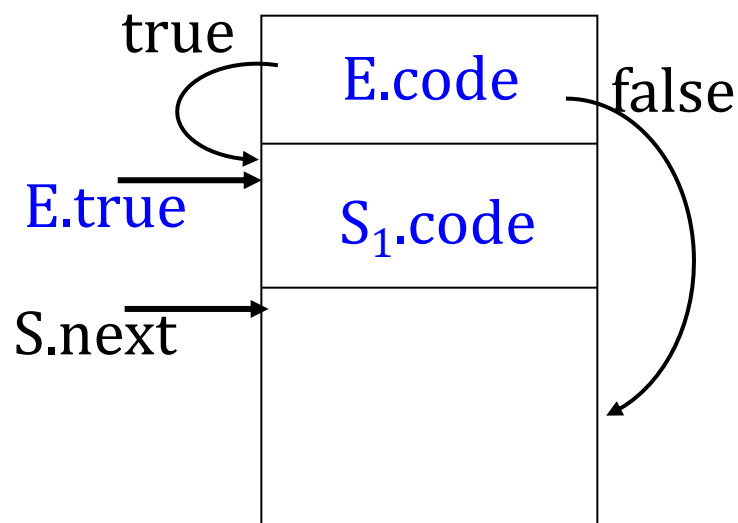


图7.17(a) if-then语句的代码结构

7.5.1 控制流语句

产生式

语义规则

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

$E.\text{true} := \text{newlabel}();$

$E.\text{false} := \text{newlabel}();$

$S_1.\text{next} := S.\text{next};$

$S_2.\text{next} := S.\text{next};$

$S.\text{code} := E.\text{code} ||$

$\text{gen}(E.\text{true} ':') ||$

$S_1.\text{code} ||$

$\text{gen}(\text{'goto' } S.\text{next}) ||$

$\text{gen}(E.\text{false} ':') ||$

$S_2.\text{code}$

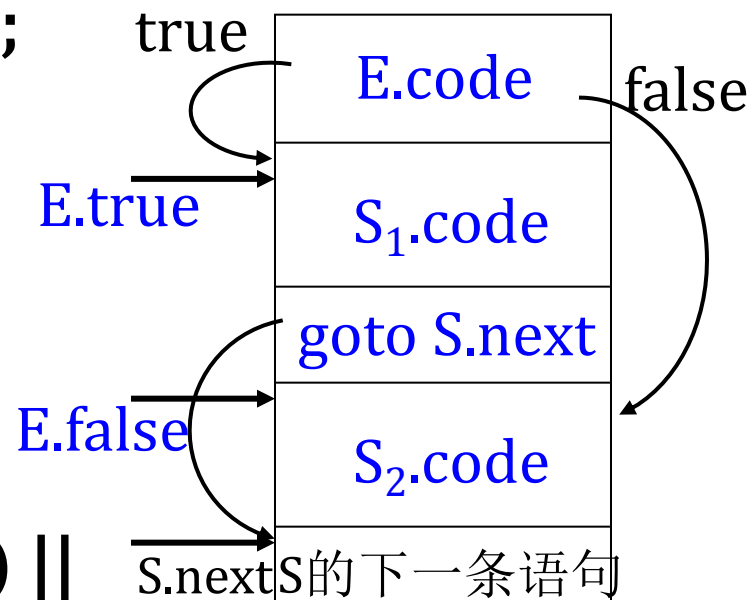


图7.17(b) if-then-else语句的代码结构

7.5.1 控制流语句

产生式

语义规则

$S \rightarrow \text{while } E \text{ do } S_1$

$S.\text{begin} := \text{newlabel}();$ $S.\text{begin}$

$E.\text{true} := \text{newlabel}();$ $\xrightarrow{\text{true}}$

$E.\text{false} := S.\text{next};$

$S_1.\text{next} := S.\text{begin};$ $E.\text{true}$

$S.\text{code} := \text{gen}(S.\text{begin} ':')$

$\parallel E.\text{code} \parallel$

$\text{gen}(E.\text{true} ':') \parallel$

$S_1.\text{code} \parallel$

$\text{gen}(\text{'goto' } S.\text{begin})$

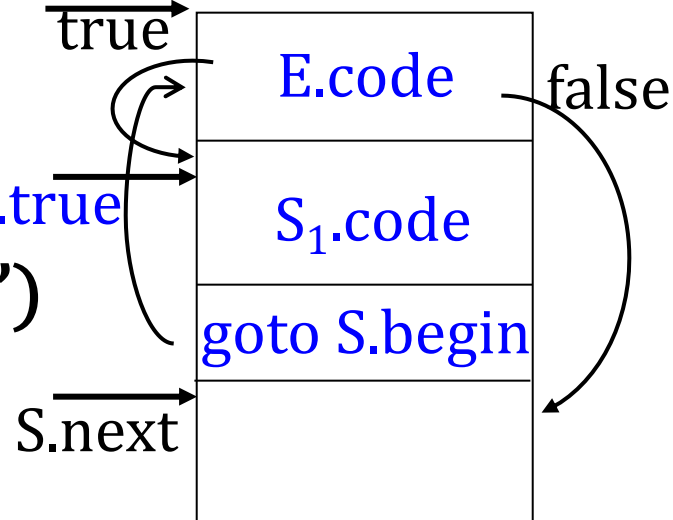


图7.17(c) while-do语句的代码结构

例7.5:

```
while a<b do
  if c<d then
    x:=y+z
  else
    x:=y-z
```

```
S→while E do S1      S.begin:=newlabel( );
                       E.true:=newlabel( );
                       E.false:=S.next;
                       S1.next:=S.begin;
                       S.code:= gen(S.begin ':') || E.code ||
                               gen(E.true ':') || S1.code || gen( 'goto' S.begin )
```

```
S→ if E then S1 else S2      E.true:=newlabel( );
                               E.false:=newlabel( );
                               S1.next:=S.next;
                               S2.next:=S.next;
                               S.code:=E.code || gen(E.true ':') ||
                               S1.code || gen( 'goto' S.next )
                               || gen(E.false ':') || S2.code
```

例7.5:

while a<b do

 if c<d then

 x:=y+z

 else

 x:=y-z

L ₁ :	if a<b goto L ₂	100	(j<, a, b, 102)
	goto L.next	101	(j, -, -, 110)
L ₂ :	if c<d goto L ₃	102	(j<, c, d, 104)
	goto L ₄	103	(j, -, -, 107)
L ₃ :	T1=y+z	104	(+, y, z, T ₁)
	x=T1	105	(assign, T ₁ , -, x)
	goto L ₁	106	(j, -, -, 100)
L ₄ :	T2=y-z	107	(-, y, z, T ₂)
	x=T2	108	(assign, T ₂ , -, x)
	goto L ₁	109	(j, -, -, 100)
L.next:		110	

- 文法G(L):
 $S \rightarrow \text{if } E \text{ then } S$
 | $\text{if } E \text{ then } S \text{ else } S$
 | $\text{while } E \text{ do } S$
 | $\text{begin } L \text{ end}$
 | A
 $L \rightarrow L;S$
 | S

这里，S表示语句，L表示语句表，
A为赋值语句，E为一个布尔表达式

- 文法

$S \rightarrow \text{if } E \text{ then } M \ S_1$

$S \rightarrow \text{if } E \text{ then } M_1 \ S_1 \ N \ \text{else } M_2 \ S_2$

$S \rightarrow \text{while } M_1 \ E \ \text{do } M_2 \ S_1$

属性:

E.truelist

E.falselist

M.quad

N.nextlist

S.nextlist

变量: nextquad

函数:

makelist

backpatch

merge

emit

转移到下一条语句的
指令链表的指针

$M \rightarrow \epsilon$ 记录变量 nextquad 的当前值, 以便回填转移到此的指令

$N \rightarrow \epsilon$ 产生一条不完整的goto指令, 并记录下它的位置

$S \rightarrow \text{if } E \text{ then } M \ S_1$
 { backpatch(E.truelist, M.quad);
 $S.\text{nextlist} = \text{merge}(E.\text{falselist}, S_1.\text{nextlist})$ }

$S \rightarrow \text{if } E \text{ then } M_1 \ S_1 \ N \ \text{else } M_2 \ S_2$
 { backpatch(E.truelist, $M_1.\text{quad}$);
 backpatch(E.falselist, $M_2.\text{quad}$);
 $S.\text{nextlist} = \text{merge}(S_1.\text{nextlist}, N.\text{nextlist}, S_2.\text{nextlist})$ }

$M \rightarrow \epsilon$

 { $M.\text{quad} = \text{nextquad}$ }

$N \rightarrow \epsilon$

 { $N.\text{nextlist} = \text{makelist}(\text{nextquad});$
 $\text{emit}('j, -, -, -')$ }

$S \rightarrow \text{while } M_1 \text{ E do } M_2 S_1$
 { backpatch($S_1.\text{nextlist}$, $M_1.\text{quad}$);
 backpatch($E.\text{truelist}$, $M_2.\text{quad}$);
 $S.\text{nextlist} = E.\text{falselist}$;
 emit('j, -, -' $M_1.\text{quad}$) }

$S \rightarrow \text{begin L end}$
 { $S.\text{nextlist} = L.\text{nextlist}$ }

$S \rightarrow A$
 { $S.\text{nextlist} = \text{makelist}()$ }

$L \rightarrow L_1; M S$
 { backpatch($L_1.\text{nextlist}$, $M.\text{quad}$);
 $L.\text{nextlist} = S.\text{nextlist}$ }

$L \rightarrow S$
 { $L.\text{nextlist} = S.\text{nextlist}$ }

例7.5:

```
while a<b do
  if c<d then
    x:=y+z
  else
    x:=y-z
```

```
100  (j<, a, b, 102)
101  (j, -, -, 110)
102  (j<, c, d, 104)
103  (j, -, -, 107)
104  (+, y, z, T1)
105  (assign, T1, -, x)
106  (j, -, -, 100)
107  (-, y, z, T2)
108  (assign, T2, -, x)
109  (j, -, -, 100)
110
```

7.6 过程调用的处理

- 过程调用主要对应两种事：
 - 传递参数
 - 转子
- 传地址:把实在参数的地址传递给相应的形式参数
 - 实在参数的地址应存放在被调用的子程序能够取得的地方；
 - 当通过转子指令进入子程序后，子程序段的第一步工作就是把实在参数的地址取到对应的形式单元中，然后，再开始执行本段中的语句；
 - 在子程序段中对形式参数的任何引用都当作是对形式单元的间接访问。

- 翻译方法：把实参的地址逐一放在转子指令的前面.

例如， CALL S(A+B, Z) 翻译为:

计算A+B置于T中的代码

```
param T    /*第一个参数的地址*/  
param z    /*第二个参数的地址*/  
call S     /*转子*/
```

过程调用的翻译

- 过程调用文法:
 - (1) $S \rightarrow \text{call id (Elist)}$
 - (2) $\text{Elist} \rightarrow \text{Elist}, E$
 - (3) $\text{Elist} \rightarrow E$
- 参数的地址存放在一个队列中
- 最后对队列队列中的每一项生成一条param语句

- 翻译模式

Elist $\rightarrow$ E

{ 初始化queue仅包含E.place }

Elist $\rightarrow$ Elist, E

{ 将E.place加入到queue的队尾 }

S $\rightarrow$ call id (Elist)

{ for 队列queue中的每一项p do
 emit('param' p);
 emit('call' id.place) }

S的代码包括:

- 1) Elist的代码(即对各参数表达式求值的代码);
- 2) 顺序为每一个参数对应一条param语句;
- 3) call语句